

Proyecto final · Consolidado del contribuyente

Guía paso a paso · qué escribir, dónde, y cómo comprobar que va bien

Cómo usar esta guía

Si tienes tiempo y quieres aprender más, **hazlo tú desde cero** con el brief (`brief/requerimientos.md`) y usa esta guía solo cuando te atasques. Aprendes bastante más así.

Si el tiempo aprieta, **síguela paso a paso**. Están todos los bloques listos para pegar y el proyecto queda funcionando. Pero lee lo que hay antes de cada bloque: en la corrección hay una parte que se lee, no solo se ejecuta, y ahí se nota quién entendió.

No necesitas Docker. Ni para desarrollar ni para el paso de la imagen: la imagen base viaja en el repositorio del curso.

Lo que hay que entregar

Un endpoint	<code>GET /consolidados/{rut}?desde=...&hasta=...</code>
Cuatro respuestas	401 sin token · 403 con rol equivocado · 404 si el RUT no existe · 200 con el consolidado
Tests	<code>./mvnw test</code> en verde
La imagen	<code>./mvnw package jib:buildTar</code>

Lo que YA está hecho — no lo escribas

Las tres entidades	<code>Contribuyente</code> , <code>Tramite</code> , <code>Oficina</code>
Los tres repositorios	Con <code>findByRut</code> y <code>findByCodigo</code> ya puestos
La seguridad entera	Login, JWT, usuarios sembrados. Solo falta una línea , en el paso 5
La base y los datos	Flyway los crea al arrancar

Los usuarios

ana / dgt2026	FISCALIZADOR	← el que Sí puede
luis / dgt2026	CONTRIBUYENTE	← el que recibe 403

El orden de trabajo · seis pasos

1	Los DTO de salida	qué se devuelve
2	La consulta en el repositorio	de dónde salen los trámites
3	El servicio	la lógica y el total
4	El controller	el endpoint
5	La regla del rol	una línea

6	El manejo de errores	el 404 y el 400 con cuerpo
---	----------------------	----------------------------

Las dos trampas del encargo. Están dichas en el brief y son donde más gente se cae:

1 · Un RUT que no existe es 404. Un contribuyente que existe y no tiene trámites en el período es 200 con la lista vacía y el total en cero. No son el mismo caso.

2 · El total NO filtra por estado. Un trámite PENDIENTE o RECHAZADO suma igual: la pregunta es cuánto se **declaró**, no cuánto se pagó.

Paso 1 · Los DTO de salida

Por qué esto va primero

Porque define **qué se devuelve**, y todo lo demás trabaja para llenarlo. Si empiezas por la consulta sin saber qué campos salen, terminas trayendo datos que nadie usa.

El brief es tajante: **estos seis campos y ninguno más**. Si devuelves la entidad directamente, salen campos que no se piden — y eso baja la nota.

Ojo con el nombre del campo. En la entidad `Tramite` el campo se llama `monto`. En el JSON tiene que salir como `montoDeclarado`. **No son el mismo nombre**, y por eso hace falta el DTO: no basta con devolver la entidad.

Qué crear

src/main/java/cl/dgt/consolidado/dto/TramiteDto.java ← archivo nuevo

```
package cl.dgt.consolidado.dto;

import java.math.BigDecimal;
import java.time.LocalDate;

public record TramiteDto(
    Long id,
    String tipo,
    String estado,
    LocalDate fecha,
    BigDecimal montoDeclarado) {
}
```

src/main/java/cl/dgt/consolidado/dto/ConsolidadoDto.java ← archivo nuevo

```
package cl.dgt.consolidado.dto;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

public record ConsolidadoDto(
    String rut,
    String razonSocial,
    LocalDate desde,
    LocalDate hasta,
    List<TramiteDto> tramites,
    BigDecimal totalDeclarado) {
}
```

Records, como en el Lab 01. Son inmutables y Jackson los serializa a JSON usando los nombres de los componentes — por eso el nombre del campo importa.

Paso 2 · La consulta en el repositorio

El problema

Hay que traer **los trámites de un contribuyente entre dos fechas**. El repositorio de trámites llega vacío: no tiene ningún método propio.

Qué agregar

```
src/main/java/cl/dgt/consolidado/repositories/TramiteRepository.java
```

```
package cl.dgt.consolidado.repositories;

import cl.dgt.consolidado.entities.Tramite;
import org.springframework.data.jpa.repository.JpaRepository;

import java.time.LocalDate;
import java.util.List;

public interface TramiteRepository extends JpaRepository<Tramite, Long> {

    List<Tramite> findByContribuyenteRutAndFechaBetweenOrderByFecha(
        String rut, LocalDate desde, LocalDate hasta);
}
```

Qué dice ese nombre — léelo por trozos

<code>findBy</code>	esto es un SELECT
<code>ContribuyenteRut</code>	navega la relación: el campo <code>rut</code> de la propiedad <code>contribuyente</code> . Spring genera el JOIN solo
<code>And</code>	une las dos condiciones
<code>FechaBetween</code>	<code>fecha between ? and ?</code> , con los dos extremos incluidos
<code>OrderByFecha</code>	ordena por fecha ascendente

Es lo del **Lab 04** (consultas derivadas) y lo del **Lab 05** (navegar una relación por el nombre del método), juntos.

Si prefieres escribir la consulta a mano, esto es equivalente y también vale:

```
@Query("""
    select t from Tramite t
    where t.contribuyente.rut = :rut
    and t.fecha between :desde and :hasta
    order by t.fecha
    """)
List<Tramite> delPeriodo(@Param("rut") String rut,
    @Param("desde") LocalDate desde,
    @Param("hasta") LocalDate hasta);
```

Paso 3 · El servicio

Qué hace

Cuatro cosas, en orden:

1	Busca el contribuyente por RUT. Si no está → 404
2	Trae sus trámites del período
3	Los convierte a DTO
4	Suma el total

Qué crear

src/main/java/cl/dgt/consolidado/exceptions/ContribuyenteNoEncontradoException.java ← nuevo

```
package cl.dgt.consolidado.exceptions;

public class ContribuyenteNoEncontradoException extends RuntimeException {

    private final String rut;

    public ContribuyenteNoEncontradoException(String rut) {
        super("No existe el contribuyente " + rut);
        this.rut = rut;
    }

    public String getRut() {
        return rut;
    }
}
```

Fíjate en que no menciona el 404 por ninguna parte. Es lo del Lab 03: la excepción describe lo que pasó en el negocio; traducirlo a HTTP es trabajo de otra pieza, la del paso 6.

src/main/java/cl/dgt/consolidado/services/ConsolidadoService.java ← nuevo

```

package cl.dgt.consolidado.services;

import cl.dgt.consolidado.dto.ConsolidadoDto;
import cl.dgt.consolidado.dto.TramiteDto;
import cl.dgt.consolidado.entities.Contribuyente;
import cl.dgt.consolidado.entities.Tramite;
import cl.dgt.consolidado.exceptions.ContribuyenteNoEncontradoException;
import cl.dgt.consolidado.repositories.ContribuyenteRepository;
import cl.dgt.consolidado.repositories.TramiteRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

@Service
public class ConsolidadoService {

    private final ContribuyenteRepository contribuyentes;
    private final TramiteRepository tramites;

    public ConsolidadoService(ContribuyenteRepository contribuyentes, TramiteRepository
tramites) {
        this.contribuyentes = contribuyentes;
        this.tramites = tramites;
    }

    @Transactional(readOnly = true)
    public ConsolidadoDto de(String rut, LocalDate desde, LocalDate hasta) {

        // 1 · el contribuyente tiene que existir. Si no, 404.
        Contribuyente contribuyente = contribuyentes.findByRut(rut)
            .orElseThrow(() -> new ContribuyenteNoEncontradoException(rut));

        // 2 · sus trámites del período. Puede venir vacío, y eso NO es un error.
        List<Tramite> delPeriodo =
            tramites.findByContribuyenteRutAndFechaBetweenOrderByFecha(rut, desde,
hasta);

        // 3 · a DTO. Aquí `monto` pasa a llamarse `montoDeclarado`.
        List<TramiteDto> lista = delPeriodo.stream()
            .map(t -> new TramiteDto(
                t.getId(), t.getTipo(), t.getEstado(), t.getFecha(), t.getMonto()))
            .toList();

        // 4 · el total. Sin filtrar por estado: suman todos.
        BigDecimal total = delPeriodo.stream()
            .map(Tramite::getMonto)
            .reduce(BigDecimal.ZERO, BigDecimal::add);

        return new ConsolidadoDto(
            contribuyente.getRut(), contribuyente.getRazonSocial(),
            desde, hasta, lista, total);
    }
}

```

Los dos puntos donde se cae la gente, y están en este archivo:

- El `orElseThrow` del paso 1 es lo que produce el 404. Si en vez de eso devuelves una lista vacía, el RUT inexistente daría 200 y **eso está mal**.

- La suma del paso 4 **no lleva ningún filtro**. Si agregas `.filter(t -> t.getEstado().equals("PAGADO"))`, el total no va a cuadrar con el esperado.

El atajo que parece más corto y da el total mal. La entidad `Contribuyente` tiene un `getTramites()`, y es tentador usarlo en vez de la consulta del paso 2.

No lo hagas: trae los trámites **de todos los años**, sin filtrar por fecha. Medido para `76.111.111-1`: devuelve **6 trámites donde el consolidado de 2026 pide 4**, con HTTP 200 y sin una sola línea de aviso.

El total sale mal y todo parece funcionar. Usa el método del repositorio.

El `@Transactional(readOnly = true)` hace falta: la relación contribuyente es LAZY y sin transacción abierta, tocarla revienta con `LazyInitializationException`. Es lo del Lab 05.

Paso 4 · El controller

Qué crear

src/main/java/cl/dgt/consolidado/controllers/ConsolidadoController.java ← nuevo

```
package cl.dgt.consolidado.controllers;

import cl.dgt.consolidado.dto.ConsolidadoDto;
import cl.dgt.consolidado.services.ConsolidadoService;
import org.springframework.format.annotation.DateTimeFormat;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import java.time.LocalDate;

@RestController
public class ConsolidadoController {

    private final ConsolidadoService servicio;

    public ConsolidadoController(ConsolidadoService servicio) {
        this.servicio = servicio;
    }

    @GetMapping("/consolidados/{rut}")
    public ConsolidadoDto consolidado(
        @PathVariable String rut,
        @RequestParam @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate desde,
        @RequestParam @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate hasta) {

        return servicio.de(rut, desde, hasta);
    }
}
```

Las tres cosas que hay que entender de aquí

@PathVariable	El RUT va en la ruta porque identifica el recurso. Lab 01
@RequestParam	Las fechas van después del ? porque filtran , no identifican. Y son obligatorias por defecto: si falta una, Spring devuelve 400 solo
@DateTimeFormat	Convierte el texto 2026-01-01 a <code>LocalDate</code> . Sin esto no compila el binding y las fechas llegan mal

El método **no atrapa ninguna excepción** y no devuelve `ResponseEntity`. Devuelve el DTO y ya. Los códigos de error los pone la pieza del paso 6 — así el controller queda limpio.

Paso 5 · La regla del rol · una línea

El problema

Ahora mismo, cualquier usuario autenticado puede pedir el consolidado. Y el brief dice **solo FISCALIZADOR**: con el token de `luis` tiene que salir **403**.

Qué cambiar

```
src/main/java/cl/dgt/consolidado/seguridad/SeguridadConfig.java
```

Busca este comentario, que ya está en el archivo:

```
// Las reglas del encargo van aquí.  
.anyRequest().authenticated()
```

Y déjalo así:

```
.requestMatchers("/consolidados/**").hasRole("FISCALIZADOR")  
.anyRequest().authenticated()
```

El orden importa, y es el error clásico. La línea del rol va **ANTES** de `anyRequest()`.

Las reglas se evalúan de arriba abajo y gana la primera que calza. Si `anyRequest()` fuera primero, se llevaría todo y a la del rol **nunca se llegaría** — y lo peor es que no falla ni avisa: simplemente cualquier autenticado entraría.

NO uses `@PreAuthorize` en el controller. Es lo que uno haría por instinto, y en este proyecto **no funciona**: falta `@EnableMethodSecurity`, así que la anotación **se ignora entera**.

Y lo peor es cómo falla: **sin error, sin warning, sin nada**. La aplicación arranca, todo parece bien, y un usuario CONTRIBUYENTE recibe **200**. El endpoint queda abierto y nadie te avisa.

Está comprobado. La regla va en la cadena, como arriba.

Por qué `hasRole("FISCALIZADOR")` y no `"ROLE_FISCALIZADOR"`

Porque `hasRole` le antepone `ROLE_` solo. Si escribes el prefijo, queda `ROLE_ROLE_FISCALIZADOR` y no calza nunca. Es lo del Lab 09.

Paso 6 · El manejo de errores

El problema

Sin esto, la excepción del paso 3 sale como **500 con una traza**. Y el brief pide **404 con un cuerpo** que diga `{"mensaje": "..."}.`

Qué crear

src/main/java/cl/dgt/consolidado/dto/ErrorRespuesta.java ← nuevo

```
package cl.dgt.consolidado.dto;

public record ErrorRespuesta(String mensaje) {
}
```

src/main/java/cl/dgt/consolidado/exceptions/ManejadorDeErrores.java ← nuevo

```
package cl.dgt.consolidado.exceptions;

import cl.dgt.consolidado.dto.ErrorRespuesta;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MissingServletRequestParameterException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class ManejadorDeErrores {

    @ExceptionHandler(ContribuyenteNoEncontradoException.class)
    public ResponseEntity<ErrorRespuesta> noEncontrado(ContribuyenteNoEncontradoException e)
    {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ErrorRespuesta(e.getMessage()));
    }

    @ExceptionHandler(MissingServletRequestParameterException.class)
    public ResponseEntity<ErrorRespuesta>
    faltaUnParametro(MissingServletRequestParameterException e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
            .body(new ErrorRespuesta("Falta el parámetro obligatorio: " +
            e.getParameterName()));
    }
}
```

Es exactamente lo del **Lab 03**. Y ahora se ve para qué servía: la excepción del servicio no sabe nada de HTTP, y esta clase traduce.

El segundo manejador es el que da el **400 con cuerpo** cuando falta desde o hasta . Sin él, Spring devuelve 400 pero con su cuerpo por defecto — y el brief pide el mensaje .

Paso 7 · Los dos tests · esto también se entrega

Esto no es opcional. El encargo pide tres tests: el de contexto que ya viene, y estos dos. Sin ellos el criterio de pruebas queda **insuficiente**, y el umbral para aprobar es suficiente en **todos** los criterios.

Si corres `./mvnw test` hoy vas a ver **Tests run: 1**. Ese es el que ya venía.

Test 1 · el servicio, con Mockito

Comprueba **el total**, sin base de datos: el repositorio se dobla. Es el Lab 08.

```
src/test/java/cl/dgt/consolidado/ConsolidadoServiceTest.java ← nuevo
```

```

package cl.dgt.consolidado;

import cl.dgt.consolidado.dto.ConsolidadoDto;
import cl.dgt.consolidado.entities.Contribuyente;
import cl.dgt.consolidado.entities.Tramite;
import cl.dgt.consolidado.exceptions.ContribuyenteNoEncontradoException;
import cl.dgt.consolidado.repositories.ContribuyenteRepository;
import cl.dgt.consolidado.repositories.TramiteRepository;
import cl.dgt.consolidado.services.ConsolidadoService;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.eq;
import static org.mockito.Mockito.when;

@ExtendWith(MockitoExtension.class)
class ConsolidadoServiceTest {

    private static final LocalDate DESDE = LocalDate.of(2026, 1, 1);
    private static final LocalDate HASTA = LocalDate.of(2026, 12, 31);

    @Mock private ContribuyenteRepository contribuyentes;
    @Mock private TramiteRepository tramites;

    @Test
    void elTotalSumaLosMontosDelPeriodo() {

        when(contribuyentes.findByRut("76.111.111-1")).thenReturn(
            Optional.of(new Contribuyente("76.111.111-1", "Comercial Andes Ltda.")));

        when(tramites.findByContribuyenteRutAndFechaBetweenOrderByFecha(
            eq("76.111.111-1"), any(), any())).thenReturn(List.of(
            tramite(1L, "F29", "PAGADO", new BigDecimal("1200000.00")),
            tramite(2L, "F22", "PENDIENTE", new BigDecimal("3400000.00"))));

        ConsolidadoService servicio = new ConsolidadoService(contribuyentes, tramites);
        ConsolidadoDto resultado = servicio.de("76.111.111-1", DESDE, HASTA);

        assertEquals(0, new BigDecimal("4600000.00").compareTo(resultado.totalDeclarado()));
        assertEquals(2, resultado.tramites().size());
    }

    @Test
    void unContribuyenteQueNoExisteLanzaLaExcepcionDel404() {

        when(contribuyentes.findByRut("99.999.999-9")).thenReturn(Optional.empty());

        ConsolidadoService servicio = new ConsolidadoService(contribuyentes, tramites);

        assertThrows(ContribuyenteNoEncontradoException.class,
            () -> servicio.de("99.999.999-9", DESDE, HASTA));
    }
}

```

```
private static Tramite tramite(Long id, String tipo, String estado, BigDecimal monto) {  
    return new Tramite(id, tipo, estado, LocalDate.of(2026, 3, 1), monto, "SCL-CEN");  
}  
}
```

Las dos trampas de este test, y las dos cuestan tiempo si no las sabes:

1 · compareTo, no equals, para comparar BigDecimal. equals compara también la **escala**: para él, 4600000.00 y 4600000.0 **no son iguales** aunque valgan lo mismo. Con equals el test se pone rojo sin motivo.

2 · Los dos trámites suman, incluido el PENDIENTE. Este test es lo que deja escrito que el total no filtra por estado: si alguien añade un if, se pone rojo.

El constructor de seis argumentos de Tramite **existe solo para los tests**, y está puesto a propósito en la entidad. La alternativa sería levantar la base o usar reflexión, y las dos son peores.

Test 2 · el controller, con MockMvc

Comprueba el **404 con cuerpo**, sin levantar el servidor.

src/test/java/cl/dgt/consolidado/ConsolidadoControllerTest.java ← nuevo

```

package cl.dgt.consolidado;

import cl.dgt.consolidado.controllers.ConsolidadoController;
import cl.dgt.consolidado.exceptions.ContribuyenteNoEncontradoException;
import cl.dgt.consolidado.exceptions.ManejadorDeErrores;
import cl.dgt.consolidado.services.ConsolidadoService;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.webmvc.test.autoconfigure.AutoConfigureMockMvc;
import org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest;
import org.springframework.context.annotation.Import;
import org.springframework.security.test.context.support.WithMockUser;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

import java.time.LocalDate;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.eq;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@WebMvcTest(controllers = ConsolidadoController.class)
@Import(ManejadorDeErrores.class)
@AutoConfigureMockMvc(addFilters = false)
@WithMockUser(authorities = "ROLE_FISCALIZADOR")
class ConsolidadoControllerTest {

    @Autowired private MockMvc mockMvc;

    @MockitoBean private ConsolidadoService servicio;

    @Test
    void unRutQueNoExisteDevuelve404ConCuerpo() throws Exception {

        when(servicio.de(eq("99.999.999-9"), any(LocalDate.class), any(LocalDate.class)))
            .thenReturn(new ContribuyenteNoEncontradoException("99.999.999-9"));

        mockMvc.perform(get("/consolidados/99.999.999-9")
            .param("desde", "2026-01-01")
            .param("hasta", "2026-12-31"))
            .andExpect(status().isNotFound())
            .andExpect(jsonPath("$.mensaje").value("No existe el contribuyente
99.999.999-9"));
    }
}

```

Las tres anotaciones de arriba son el test, y sin ellas no funciona:

`@Import(ManejadorDeErrores.class)` — `@WebMvcTest` carga **solo** el controller que se le nombra. Sin este import, el manejador no está, la excepción se escapa como 500 y el test falla.

`addFilters = false` — apaga la cadena de seguridad **en este test**. Lo que se prueba aquí es el 404 y su cuerpo, no la seguridad. **Y sin esto, `@WithMockUser` no basta: la petición devuelve 401** y te vas a volver loco buscando por qué.

La seguridad se comprueba aparte, con los `curl` de la rúbrica. Cada test al nivel más barato que responda su pregunta — Lab 08.

Comprueba

```
./mvnw test
```

Ahora tienen que salir **los tres**: el de contexto, el de servicio y el de controller.

Comprobar que está bien · antes de entregar

1 · Arranca la aplicación

```
cd base
./mvnw spring-boot:run
```

Puerto **8107**. Si dice que está ocupado, el mensaje trae el comando para liberarlo.

2 · Los cuatro `curl` · son los mismos de la corrección

En otra terminal:

```
# el token de ana, que es FISCALIZADOR
TOKEN=$(curl -s -X POST localhost:8107/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"usuario":"ana","clave":"dgt2026"}' | sed 's/.*"token": "\([^"]*\)".*/\1/')

# 1 · sin token -> 401
curl -s -o /dev/null -w "%{http_code}\n" \
  "localhost:8107/consolidados/76.111.111-1?desde=2026-01-01&hasta=2026-12-31"

# 2 · token de CONTRIBUYENTE -> 403
TC=$(curl -s -X POST localhost:8107/auth/login -H 'Content-Type: application/json' \
  -d '{"usuario":"luis","clave":"dgt2026"}' | sed 's/.*"token": "\([^"]*\)".*/\1/')
curl -s -o /dev/null -w "%{http_code}\n" -H "Authorization: Bearer $TC" \
  "localhost:8107/consolidados/76.111.111-1?desde=2026-01-01&hasta=2026-12-31"

# 3 · RUT que no existe -> 404 con cuerpo
curl -s -w " [%{http_code}]\n" -H "Authorization: Bearer $TOKEN" \
  "localhost:8107/consolidados/99.999.999-9?desde=2026-01-01&hasta=2026-12-31"

# 4 · el caso bueno -> 200
curl -s -H "Authorization: Bearer $TOKEN" \
  "localhost:8107/consolidados/76.111.111-1?desde=2026-01-01&hasta=2026-12-31"
```

El número que tiene que dar el cuarto: `totalDeclarado` = **6330000**

Si te da otro, casi seguro estás filtrando por estado. La suma incluye **todos** los trámites del período.

3 · Los dos casos extra · suben la nota

```
# contribuyente que existe pero SIN trámites -> 200, lista vacía, total 0
curl -s -H "Authorization: Bearer $TOKEN" \
  "localhost:8107/consolidados/78.333.333-3?desde=2026-01-01&hasta=2026-12-31"

# falta un parámetro -> 400 con cuerpo
curl -s -w " [%{http_code}]\n" -H "Authorization: Bearer $TOKEN" \
  "localhost:8107/consolidados/76.111.111-1?desde=2026-01-01"
```

4 · Los tests y la imagen

```
./mvnw test
./mvnw package jib:buildTar
ls -lh target/jib-image.tar
```

No hace falta Docker. Jib construye el tar sin ningún demonio, y la imagen base ya está en el repositorio del curso.

5 · Antes de mandarlo

```
cd base
rm -rf target .datos-pg
```

Para que el zip no lleve tu base local ni los compilados.

Lo que se corrige

1	<code>./mvnw test</code> en verde
2	Los cuatro <code>curl</code> : 401, 403, 404 y 200 con el total correcto
3	<code>./mvnw package jib:buildTar</code>
4	Y lo que se lee : que las capas estén separadas y los nombres se entiendan

Si te queda tiempo, lo que más suma en el criterio que se lee: agregar un par de tests propios del servicio —el caso feliz y el del contribuyente sin trámites— con Mockito, como en el Lab 08.